SOFTWARE ENGINEERING · CHAPTER 12

Deployment Diagrams

Deployment diagrams model the physical arrangement of execution environments, nodes, artifacts, and communication paths. They connect software architecture to infrastructure and operations.

UML 2.5.1 **JDK 25 LTS · Spring Boot 4.1** **Docker · Kubernetes**

AGENDA

What we will cover

1. Nodes, devices, and execution environments
2. Artifacts and deployment specifications
3. Communication paths and protocols
4. Application servers, databases, and messaging systems
5. Network zones, firewalls, and load balancers
6. Containerized and cloud deployments
7. Scalability, availability, and failover
8. Configuration and environment differences
9. Security boundaries and monitoring
10. Relating deployment views to component architecture
11. Chapter Quiz
12. Lab-12

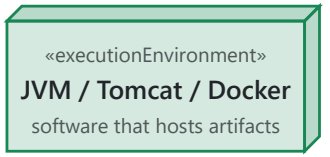
Click any item to jump directly to that topic.

From "it runs on my laptop" to "it runs for 50 000 customers"

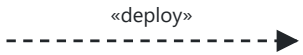
- **Component diagrams** (Ch. 11) answer *what* the software is made of. **Deployment diagrams** answer *where* it runs and *how the pieces talk*.
- Vocabulary of UML 2.5.1: **node** (3D box), **artifact** (file with a document icon), **communication path** (solid line), **«deploy»** and **«manifest»** (dashed arrows).
- Readers: operations, security, cloud cost, on-call engineers. The diagram is the shared map for incidents and capacity planning.
- Running example: the **online shop** from Chapters 02-06, first on one VM, then behind a load balancer, finally on Kubernetes.

> Mermaid has no deployment diagram. In this chapter every diagram is inline SVG and its **PlantUML** source is shown beside it so you can reproduce it.

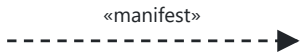








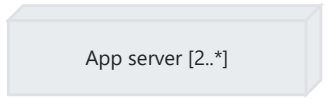
artifact → node (or draw it inside)



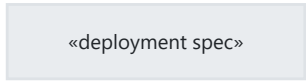
artifact → component it embodies



communication path (solid, labelled)



multiplicity = number of instances

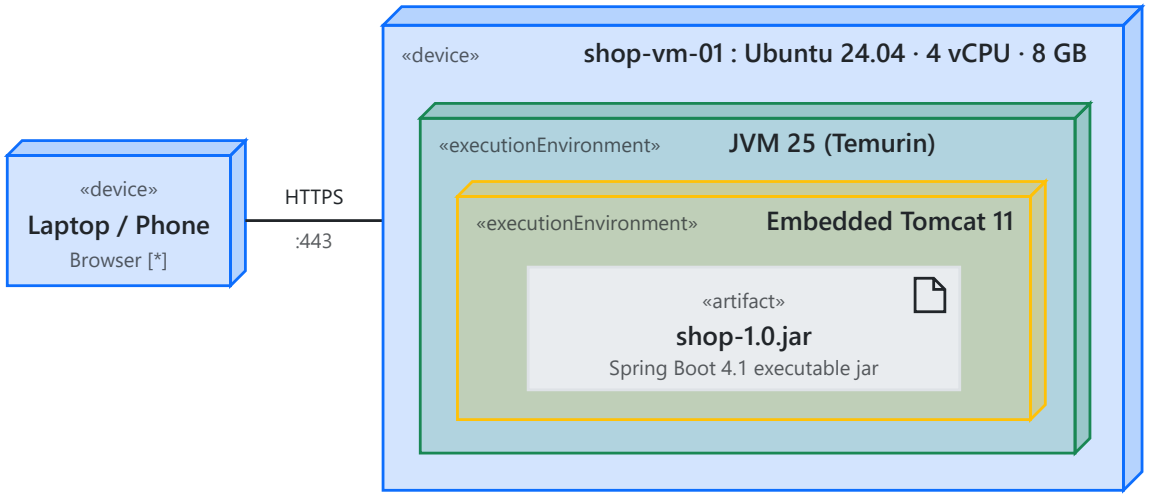


application-prod.yml, env vars



systems you do not operate

Nodes: devices contain execution environments contain artifacts



- **Node**: a computational resource on which artifacts are deployed. Drawn as a 3D box.
- **«device»**: physical or virtual hardware (server, VM, phone, router). May contain other devices.
- **«executionEnvironment»**: software that provides services for the artifacts it hosts: OS, JVM, servlet container, Docker engine, database engine.
- **Nesting** shows containment: the jar runs *in* Tomcat, *in* the JVM, *on* the VM.

```
@startuml
node "Laptop / Phone" <<device>> as client
node "shop-vm-01" <<device>> {
  node "JVM 25" <<executionEnvironment>> {
    node "Embedded Tomcat 11" <<executionEnvironment>> {
      artifact "shop-1.0.jar"
    }
  }
}
client -- "shop-vm-01" : HTTPS :443
@enduml
```

Choosing the right node kind, and saying how many there are

Stereotype	Use for	Shop example
`«device»`	Hardware or a VM that behaves like hardware	`shop-vm-01`, customer phone, load balancer appliance
`«executionEnvironment»`	Runtime that hosts artifacts	JVM 25, Tomcat 11, Docker Engine, PostgreSQL 17 server
`«container»` (profile)	OCI container running an image	`shop:1.0` container in a pod
`«cloud»` / `«external»`	Systems you rent or do not control	Stripe API, CDN, managed PostgreSQL
`«database»`, `«queue»`	Common PlantUML shapes for data stores and brokers	PostgreSQL primary, RabbitMQ

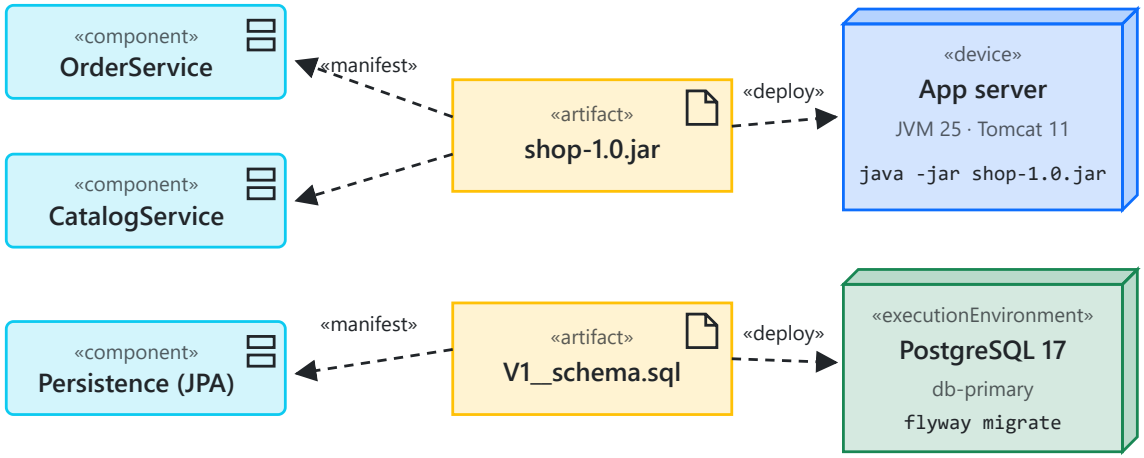
⚠ A JVM is *not* a device and a VM is *not* an execution environment. Ask "is it hardware-like (I can reboot it)?" → device; "does it run my files?" → execution environment.

```
@startuml
' multiplicity in the name: how many instances exist
node "App server [2..*]" <<device>> as app {
  node "JVM 25" <<executionEnvironment>> {
    artifact "shop-1.0.jar"
  }
}
node "Load balancer" <<device>> as lb
database "PostgreSQL 17" as db
lb -- app : HTTP :8080
app -- db : JDBC :5432
@enduml
```

```
# What the "JVM 25" execution environment actually runs
$ java -version
openjdk version "25" 2025-09-16 LTS
$ java -XX:MaxRAMPercentage=75 -jar shop-1.0.jar \
       --spring.profiles.active=prod
...  Tomcat started on port 8080 (http) with context path '/'
...  Started ShopApplication in 3.812 seconds
```

- Multiplicity `[2..*]` on a node means "at least two of these"; it replaces drawing the same box twice.
- Give nodes real names (`shop-vm-01`, `db-primary`) and sizing when relevant: readers plan capacity from the diagram.

Artifacts: the files that *manifest* components and are *deployed* on nodes



manifestation: component ⇐ artifact · deployment: artifact ⇒ node

- **Artifact**: a physical piece of information used or produced by development: jar, war, Docker image, SQL script, config file, PDF.
- **«manifest»**: the artifact is the concrete embodiment of one or more components (many components in one jar is normal).
- **«deploy»**: the artifact is installed on a node. Draw the artifact *inside* the node instead of the arrow when space is tight.
- Artifacts can depend on each other: `shop-1.0.jar` «uses» `postgresql-42.7.jar`.

```
@startuml
component "OrderService" as os
component "CatalogService" as cs
artifact "shop-1.0.jar" as jar
artifact "V1__schema.sql" as sql
node "App server" <<device>> as app
database "PostgreSQL 17" as db

jar ..> os : <<manifest>>
jar ..> cs : <<manifest>>
jar ..> app : <<deploy>>
sql ..> db  : <<deploy>>
@enduml
```

Deployment specification: the parameters that make the same jar behave differently

In UML a **deployment specification** is a box attached to a deployed artifact listing its execution parameters. For a Spring Boot jar that box is `application-prod.yml` plus environment variables plus JVM flags.

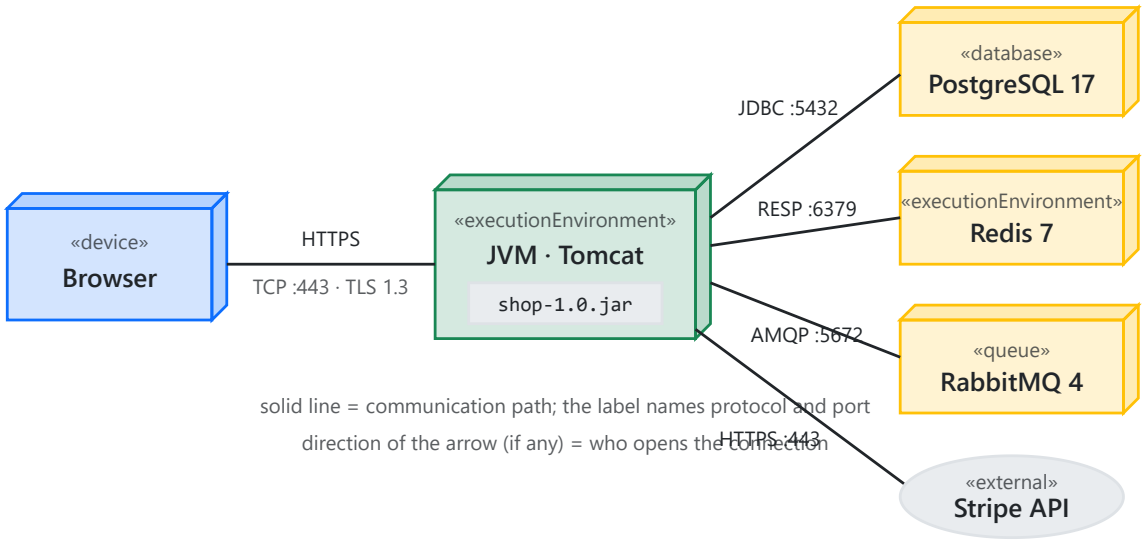
```yaml
# application-prod.yml  (packaged in the jar, secrets injected)
server:
  port: 8080
  tomcat:
    threads:
      max: 200          # concurrent request threads
    max-connections: 8192
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 20   # per instance, DB has 100 max
      minimum-idle: 5
      connection-timeout: 3000
  jpa:
    open-in-view: false
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
  endpoint:
    health:
      probes:
        enabled: true     # /actuator/health/liveness + readiness
```

```bash
# Environment variables = deployment spec supplied by the node
export SPRING_PROFILES_ACTIVE=prod
export DB_URL=jdbc:postgresql://db-primary:5432/shop
export DB_USER=shop
export DB_PASSWORD=$(cat /run/secrets/db_password)
export JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 \
  -XX:+UseZGC -Djava.security.egd=file:/dev/./urandom"
java -jar /opt/shop/shop-1.0.jar
```

```
@startuml
node "App server" <<device>> {
  artifact "shop-1.0.jar" as jar
  artifact "application-prod.yml" <<deployment spec>> as spec
  spec ..> jar
}
note right of spec
  SPRING_PROFILES_ACTIVE=prod
  server.tomcat.threads.max=200
  hikari.maximum-pool-size=20
  -XX:MaxRAMPercentage=75
end note
@enduml
```

Parameter	Rule of thumb
`threads.max`	200 default; raise only with measurements
Hikari pool	instances × pool ≤ DB `max_connections` − 10
`MaxRAMPercentage`	75 % of the container limit, never a fixed `-Xmx`

Communication paths: who talks to whom, over what, on which port



- A **communication path** is an association between nodes: the network link over which deployed artifacts exchange messages.
- Label it with the **application protocol** and the **port**; add transport details (TLS, mTLS) when they matter for security.
- Prefer the **logical** protocol (JDBC, AMQP) over "TCP": readers must know which firewall rule to open.
- Show **outbound calls to external systems** too; they need egress rules and time-outs.

```
@startuml
node "Browser" <<device>> as b
node "JVM / Tomcat" <<executionEnvironment>> as app {
  artifact "shop-1.0.jar"
}
database "PostgreSQL 17" as db
node "Redis 7" as redis
queue "RabbitMQ 4" as mq
cloud "Stripe API" as stripe

b   --> app    : HTTPS :443
app --> db     : JDBC :5432
app --> redis  : RESP :6379
app --> mq     : AMQP :5672
app --> stripe : HTTPS :443
@enduml
```

Protocol / port cheat sheet and how to verify a path exists

Path	Protocol	Port	Note
Browser → CDN/WAF	HTTPS (HTTP/2, HTTP/3)	443	TLS terminates here or at the LB
LB → app	HTTP	8080	plain inside the zone, or mTLS
app → PostgreSQL	JDBC (PostgreSQL wire)	5432	`sslmode=require` across zones
app → Redis	RESP	6379	6380 when TLS
app → RabbitMQ	AMQP 0-9-1	5672	5671 TLS; 15672 mgmt UI
Prometheus → app	HTTP scrape	8080	`/actuator/prometheus`
Ops → nodes	SSH	22	bastion only
Primary → replica	PostgreSQL streaming	5432	replication slot

Every solid line in the diagram becomes a firewall rule, a security-group entry or a Kubernetes NetworkPolicy. If you cannot name protocol and port, the line is not finished.

```
# 1. Is the app reachable from the load balancer host?
$ curl -fsS http://app-01:8080/actuator/health
{"status":"UP","groups":["liveness","readiness"]}

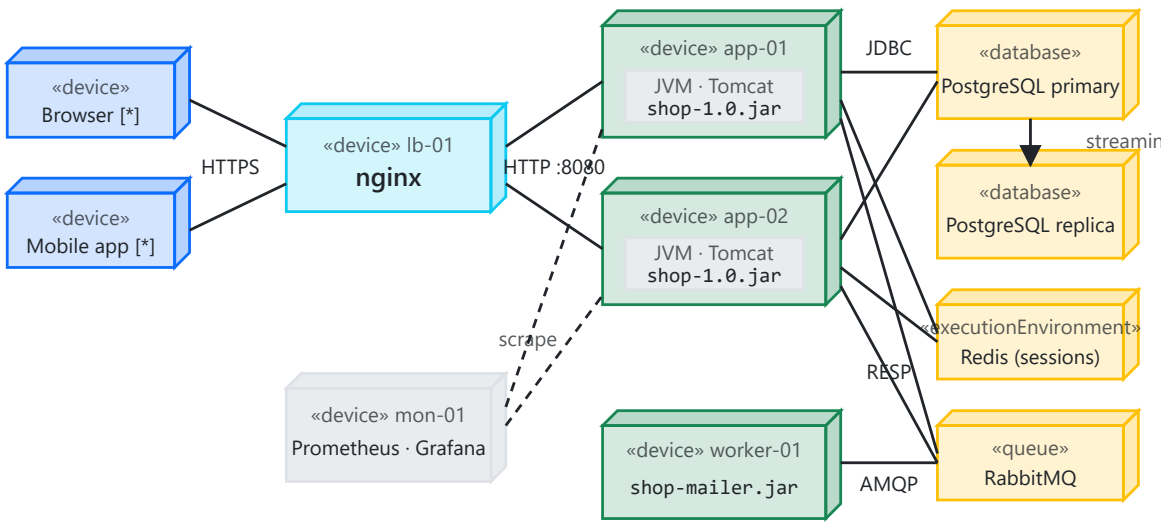
# 2. Is the JDBC path open?  (from the app host)
$ nc -zv db-primary 5432
Connection to db-primary 5432 port [tcp/postgresql] succeeded!

# 3. Readiness only becomes UP when DB and broker respond
$ curl -s http://app-01:8080/actuator/health/readiness
{"status":"UP"}

# 4. Externally, through the LB, with TLS
$ curl -sI https://shop.example.com/ | head -3
HTTP/2 200
server: nginx
strict-transport-security: max-age=31536000
```

```
// The Java side of the JDBC path: Spring Boot builds the DataSource
// from the deployment spec, HikariCP owns the TCP connections.
@Configuration
public class DataSourceCheck {
    @Bean
    ApplicationRunner verifyDb(JdbcClient jdbc) {
        return args -> jdbc.sql("select version()")
                           .query(String.class).single();
    }
}
```

The classic three-tier topology of the shop



Stripe and CDN omitted here; see topic 5 for the zones around this topology

```yaml
# docker-compose.yml: the same topology on one laptop
services:
  app:
    image: ghcr.io/itc/shop:1.0
    ports: ["8080:8080"]
    environment:
      SPRING_PROFILES_ACTIVE: prod
      DB_URL: jdbc:postgresql://db:5432/shop
      DB_USER: shop
      DB_PASSWORD: ${DB_PASSWORD}
      SPRING_DATA_REDIS_HOST: redis
      SPRING_RABBITMQ_HOST: rabbitmq
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_started }
      rabbitmq: { condition: service_started }
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: shop
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop"]
      interval: 5s
  redis:
    image: redis:7-alpine
  rabbitmq:
    image: rabbitmq:4-management
    ports: ["15672:15672"]
volumes:
  pgdata: {}
```

Making two app servers behave like one: nginx, Redis sessions, RabbitMQ

```
# /etc/nginx/conf.d/shop.conf  on lb-01
upstream shop_app {
    least_conn;
    server app-01:8080 max_fails=3 fail_timeout=10s;
    server app-02:8080 max_fails=3 fail_timeout=10s;
    keepalive 32;
}
server {
    listen 443 ssl http2;
    server_name shop.example.com;
    ssl_certificate     /etc/ssl/shop.crt;
    ssl_certificate_key /etc/ssl/shop.key;

    location / {
        proxy_pass http://shop_app;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For   $remote_addr;
        proxy_set_header X-Forwarded-Proto https;
        proxy_read_timeout 30s;
    }
    location /actuator/ { deny all; }
}
```

⚠ Two instances break in-memory HTTP sessions and `@Scheduled` jobs. Externalize state (Redis) and elect one worker for jobs, or the diagram lies about scaling.

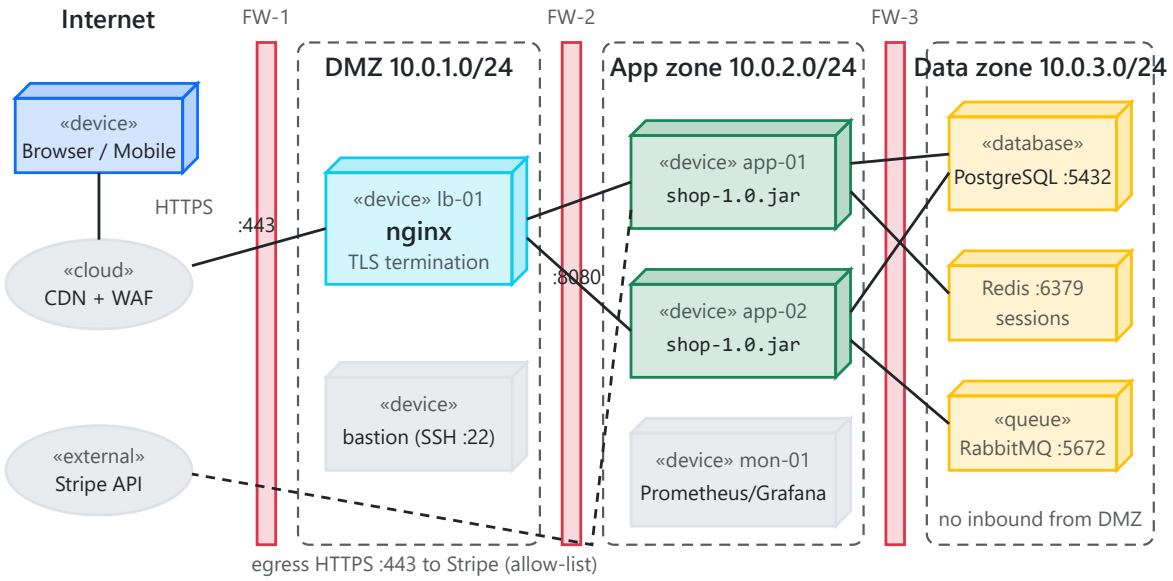
```
// Session state moves to Redis so any instance can serve any user
// (spring-session-data-redis on the classpath does the rest)
@Configuration
@EnableRedisHttpSession(maxInactiveIntervalInSeconds = 1800)
public class SessionConfig { }

// Asynchronous work goes through the broker, consumed by worker-01
@Service
public class OrderPlacedPublisher {
    private final RabbitTemplate rabbit;
    public OrderPlacedPublisher(RabbitTemplate rabbit) {
        this.rabbit = rabbit;
    }
    public void publish(OrderPlaced event) {
        rabbit.convertAndSend("shop.events", "order.placed", event);
    }
}

@Component
public class OrderMailer {
    @RabbitListener(queues = "order-mail")
    public void onOrderPlaced(OrderPlaced event) {
        // send confirmation e-mail; retries handled by the broker
    }
}
```

```
# Boot 4: trust the proxy headers so redirects stay https
server:
  forward-headers-strategy: framework
spring:
  session:
    redis:
      namespace: shop:session
```

Network zones: DMZ, application zone, data zone, and the firewalls between them

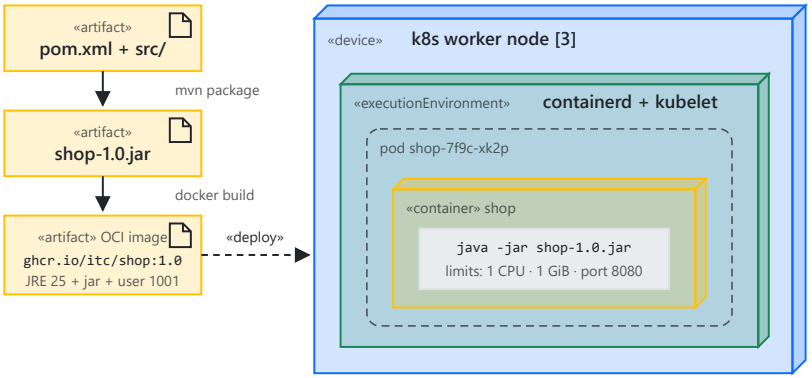


Firewall	Allows only
FW-1 Internet → DMZ	CDN/WAF → lb-01 :443; ops VPN → bastion :22
FW-2 DMZ → App	lb-01 → app-0x :8080; bastion → any :22
FW-3 App → Data	app-0x → db :5432, redis :6379, mq :5672; mon-01 → exporters
Egress	app-0x → api.stripe.com :443 only

```
@startuml
cloud "CDN + WAF" as cdn
rectangle "DMZ 10.0.1.0/24" {
  node "lb-01 (nginx)" <<device>> as lb
}
rectangle "App zone 10.0.2.0/24" {
  node "app-01" <<device>> { artifact "shop-1.0.jar" }
  node "app-02" <<device>> { artifact "shop-1.0.jar" }
}
rectangle "Data zone 10.0.3.0/24" {
  database "PostgreSQL" as db
  queue "RabbitMQ" as mq
}
cdn --> lb : HTTPS :443
lb --> "app-01" : HTTP :8080
"app-01" --> db : JDBC :5432 (FW-3)
@enduml
```

Zones are drawn as dashed rectangles (a UML *package* or a note), firewalls as named barriers. The **data zone never accepts connections from the DMZ**; a compromised load balancer must not see the database.

Containers: the image is the artifact, the container is the execution environment



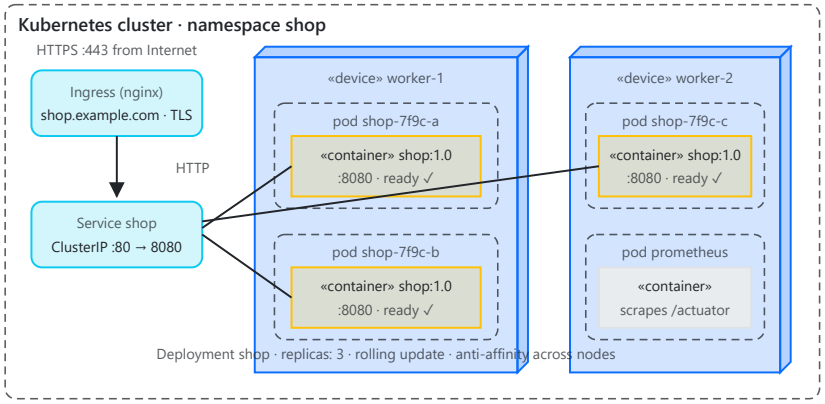

```dockerfile
# Dockerfile: multi-stage, JDK 25 to build, JRE 25 to run, non-root
FROM eclipse-temurin:25-jdk AS build
WORKDIR /src
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN ./mvnw -q dependency:go-offline
COPY src/ src/
RUN ./mvnw -q -DskipTests package

FROM eclipse-temurin:25-jre
RUN useradd --system --uid 1001 --create-home app
WORKDIR /opt/shop
COPY --from=build --chown=app:app /src/target/shop-1.0.jar shop.jar
USER 1001
EXPOSE 8080
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+UseZGC"
HEALTHCHECK --interval=15s --timeout=3s --start-period=40s \
  CMD curl -fsS http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "shop.jar"]
```

```shell
$ docker build -t ghcr.io/itc/shop:1.0 .
$ docker run --rm -p 8080:8080 -e SPRING_PROFILES_ACTIVE=dev \
    ghcr.io/itc/shop:1.0
$ docker push ghcr.io/itc/shop:1.0
```

- The **image** is an immutable artifact with a digest; the **container** is a running execution environment created from it. Draw the image where you would draw the jar.
- `MaxRAMPercentage` makes the JVM respect the container memory limit; a hard-coded `-Xmx` would ignore it.
- Kubernetes probes replace the Docker `HEALTHCHECK`; keep the latter for plain Docker and Compose.

Kubernetes objects on the deployment diagram: the Deployment



The probes call the Actuator health groups from Chapter 02; Kubernetes is just another client of `/actuator/health`.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: shop, namespace: shop }
spec:
  replicas: 3
  selector: { matchLabels: { app: shop } }
  strategy:
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  template:
    metadata: { labels: { app: shop } }
    spec:
      containers:
      - name: shop
        image: ghcr.io/itc/shop:1.0
        ports: [{ containerPort: 8080 }]
        envFrom:
        - configMapRef: { name: shop-config }
        - secretRef:    { name: shop-secrets }
        resources:
          requests: { cpu: 500m, memory: 768Mi }
          limits:   { cpu: "1",  memory: 1Gi }
        readinessProbe:
          httpGet: { path: /actuator/health/readiness, port: 8080 }
          initialDelaySeconds: 10
          periodSeconds: 5
        livenessProbe:
          httpGet: { path: /actuator/health/liveness, port: 8080 }
          initialDelaySeconds: 30
          periodSeconds: 10
          failureThreshold: 3
```

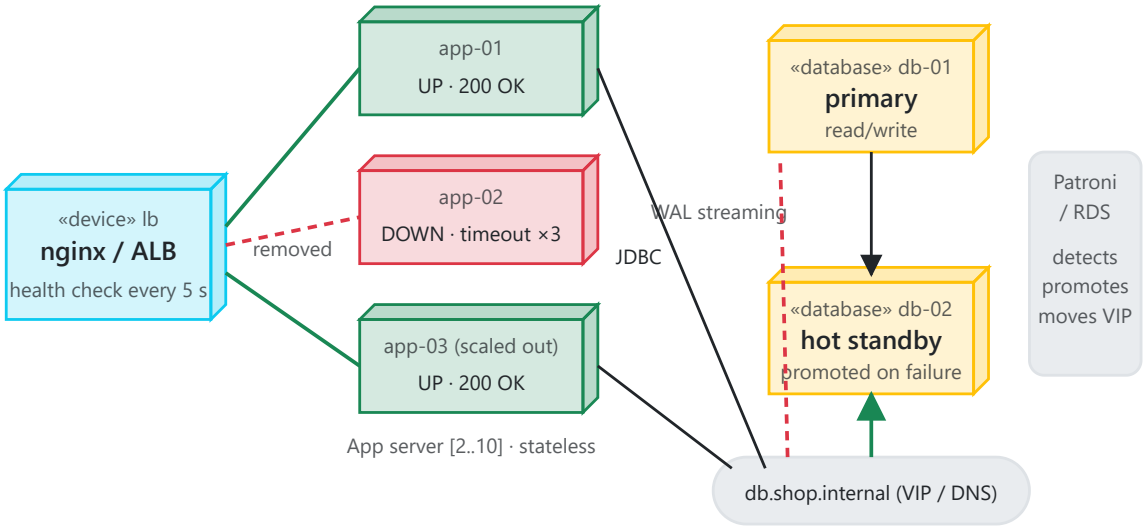
Service and Ingress, and how each Kubernetes object reads as UML

```yaml
apiVersion: v1
kind: Service
metadata: { name: shop, namespace: shop }
spec:
  selector: { app: shop }
  ports:
  - port: 80
    targetPort: 8080
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: shop
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt
spec:
  tls:
  - hosts: [shop.example.com]
    secretName: shop-tls
  rules:
  - host: shop.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service: { name: shop, port: { number: 80 } }
```

Kubernetes object	UML deployment element
Worker node	«device»
containerd / kubelet	«executionEnvironment» on the device
Pod	nested node grouping one or more containers
Container image	«artifact» (deployed into the container)
Deployment manifest, ConfigMap, Secret	deployment specification of the artifact
Service	named communication path (stable address) to the pods
Ingress	entry node of the cluster: TLS termination, HTTPS :443 path
Namespace, NetworkPolicy	zone / trust boundary

```
$ kubectl -n shop apply -f k8s/
$ kubectl -n shop get pods -o wide
NAME                READY   STATUS    NODE
shop-7f9c-a         1/1     Running   worker-1
shop-7f9c-b         1/1     Running   worker-1
shop-7f9c-c         1/1     Running   worker-2
$ kubectl -n shop rollout status deployment/shop
deployment "shop" successfully rolled out
```

Scaling out, staying up: replicas, health checks and failover on the diagram



horizontal scaling: add app nodes · failover: health check removes a node, standby is promoted

Tier	Topology	Typical availability
Single instance	1 VM, app + DB together	≈ 99 % · minutes of downtime per deploy
Active / passive	1 live app, 1 cold or warm standby; DB with replica	≈ 99.9 % · failover in minutes
Active / active	LB + n stateless apps, DB primary + hot standby, auto failover	≈ 99.95 - 99.99 % · seconds
Multi-region	Active/active in two regions, global DNS	> 99.99 % · survives a region loss

- **Scalability** is only honest if the node is stateless: sessions in Redis, files in object storage, jobs on the broker.
- **Availability** is the product of the tiers: two 99.9 % tiers in series give 99.8 %. Draw the weakest link (usually the database).
- **Failover** needs a detector (health check), a decision (promote, re-route) and a stable address (VIP, DNS, Service). Show all three.
- Multiplicity `[2..10]` documents the autoscaling range.

Autoscaling rules and surviving a database failover

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: shop }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: shop
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

```yaml
# PodDisruptionBudget: never below 2 during node drains
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: shop }
spec:
  minAvailable: 2
  selector: { matchLabels: { app: shop } }
```

```yaml
# application-prod.yml: JDBC URL lists primary and standby;
# the driver connects to whichever accepts writes.
spring:
  datasource:
    url: >-
      jdbc:postgresql://db-01:5432,db-02:5432/shop
      ?targetServerType=primary&connectTimeout=3
    hikari:
      maximum-pool-size: 20
      max-lifetime: 600000       # recycle every 10 min
      validation-timeout: 2000
      keepalive-time: 30000
```

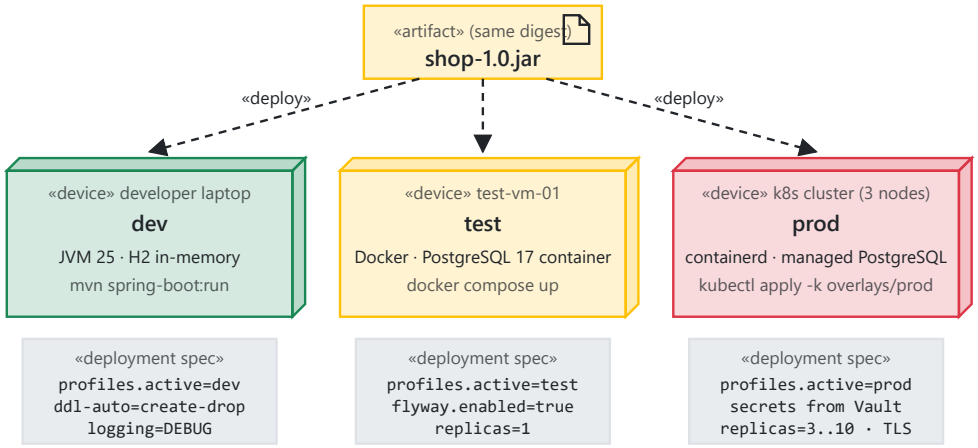
```java
// Readiness reflects the dependencies; the LB stops sending
// traffic while we cannot reach the database.
@Component
public class DatabaseReadiness implements HealthIndicator {
    private final JdbcClient jdbc;
    public DatabaseReadiness(JdbcClient jdbc) { this.jdbc = jdbc; }

    @Override
    public Health health() {
        try {
            jdbc.sql("select 1").query(Integer.class).single();
            return Health.up().build();
        } catch (DataAccessException ex) {
            return Health.down(ex).build();
        }
    }
}
```

```yaml
management:
  endpoint:
    health:
      group:
        readiness:
          include: databaseReadiness,rabbit,redis
```

💡 Liveness answers "should this process be restarted?"; readiness answers "may traffic be sent?". A DB outage must fail readiness, never liveness, or Kubernetes restarts healthy pods in a loop.

One artifact, three environments: only the deployment specification changes



	dev	test	prod
Profile	`dev`	`test`	`prod`
Nodes	laptop	1 VM, Compose	3+ k8s nodes, 2 zones
Database	H2 in-memory	PostgreSQL container	managed PostgreSQL, primary + standby
Schema	`ddl-auto=create-drop`	Flyway	Flyway, `ddl-auto=validate`
Secrets	defaults in `application-dev.yml`	`.env` file	Vault / k8s Secret, rotated
TLS	none	self-signed at nginx	cert-manager, Let's Encrypt
Instances	1	1	3..10 (HPA)
Logging	DEBUG, console	INFO, files	INFO JSON → Loki
Data	seed script	anonymised copy	real, backed up nightly

⚠ Rebuilding the jar per environment ("prod build") destroys the guarantee that what was tested is what runs. Build once, promote the same digest, vary only the specification.

Where each environment's specification lives

```yaml
# application.yml (defaults, packaged in the jar)
spring:
  application:
    name: shop
  profiles:
    active: ${SPRING_PROFILES_ACTIVE:dev}
---
# application-dev.yml
spring:
  config:
    activate:
      on-profile: dev
  datasource:
    url: jdbc:h2:mem:shop
  jpa:
    hibernate:
      ddl-auto: create-drop
  h2:
    console:
      enabled: true
logging:
  level:
    edu.itc.shop: DEBUG
---
# application-test.yml
spring:
  config:
    activate:
      on-profile: test
  datasource:
    url: jdbc:postgresql://localhost:5432/shop
  flyway:
    enabled: true
```

```yaml
apiVersion: v1
kind: ConfigMap
metadata: { name: shop-config, namespace: shop }
data:
  SPRING_PROFILES_ACTIVE: prod
  DB_URL: jdbc:postgresql://db.shop.internal:5432/shop
  SPRING_DATA_REDIS_HOST: redis.shop.svc
  SPRING_RABBITMQ_HOST: rabbitmq.shop.svc
  SERVER_TOMCAT_THREADS_MAX: "200"
---
apiVersion: v1
kind: Secret
metadata: { name: shop-secrets, namespace: shop }
type: Opaque
stringData:            # in git only as sealed-secret / from Vault
  DB_USER: shop
  DB_PASSWORD: "<injected by CI>"
  STRIPE_API_KEY: "<injected by CI>"
```

```
# Which source supplied a value?  (management port 8081)
$ kubectl -n shop exec deploy/shop -- \
    curl -s localhost:8081/actuator/env/DB_URL | jq .property
{ "source": "systemEnvironment",
  "value": "jdbc:postgresql://db.shop.internal:5432/shop" }
```

Precedence (highest first): command-line args → environment variables (ConfigMap / Secret) → `application-{profile}.yml` → `application.yml`. The diagram shows *where* each layer is stored: in the jar, on the node, in the cluster, in the vault.

The pipeline that moves one artifact through the environments

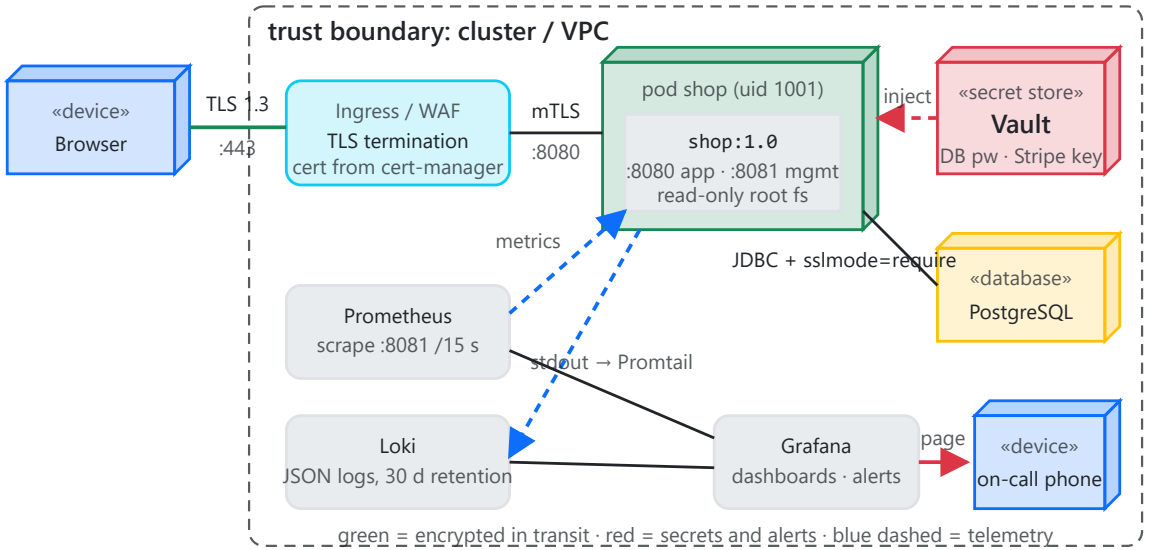
```yaml
# .github/workflows/deploy.yml  (deploy job; test + build-image run first)
deploy-prod:
  needs: [test, build-image]
  runs-on: ubuntu-latest
  environment: production          # requires manual approval
  steps:
  - uses: actions/checkout@v4
  - uses: azure/setup-kubectl@v4
  - name: kubeconfig
    run: |
      echo "${{ secrets.KUBECONFIG }}" | base64 -d > kubeconfig
      echo "KUBECONFIG=$PWD/kubeconfig" >> "$GITHUB_ENV"
  - name: create or update the secret
    run: |
      kubectl -n shop create secret generic shop-secrets \
        --from-literal=DB_USER=shop \
        --from-literal=DB_PASSWORD='${{ secrets.DB_PASSWORD }}' \
        --dry-run=client -o yaml | kubectl apply -f -
  - name: roll out the tested image digest
    run: |
      kubectl -n shop apply -k k8s/overlays/prod
      kubectl -n shop set image deployment/shop \
        shop=ghcr.io/itc/shop@${{ needs.build-image.outputs.digest }}
      kubectl -n shop rollout status deployment/shop --timeout=180s
  - name: smoke test
    run: curl -fsS https://shop.example.com/actuator/health
```

Stage	Produces / touches	UML element
`test`	runs the Chapter 06 JUnit 6 suite against PostgreSQL in Testcontainers	verifies the artifact
`build-image`	`ghcr.io/itc/shop@sha256:…`	«artifact»
`deploy-test`	Compose on test-vm-01	«deploy» to test node
`deploy-prod`	same digest, prod overlay, secrets	«deploy» + deployment spec
smoke test	`curl /actuator/health`	exercises the HTTPS path

☑ Deploy by **digest**, not by tag: `:1.0` can be overwritten, `@sha256:…` cannot. The digest is the artifact identity on the diagram.

⚠ `rollout status` waits for readiness probes; without probes the pipeline declares success while pods are still starting.

Security boundaries and observability flows on the same diagram



- **Trust boundary**: a dashed region; everything crossing it is authenticated and encrypted. Name where TLS terminates and what happens after (plain HTTP inside a private network, or mTLS via a service mesh).
- **Secrets** are artifacts too, but never drawn in git: show their *source* (Vault, k8s Secret, cloud KMS) and the path into the node.
- **Management port**: expose Actuator on `8081`, reachable only from Prometheus, never through the Ingress.
- **Three telemetry flows**: metrics (pull), logs (push), traces (push). Draw them dashed so they are not mistaken for business traffic.

```
# prometheus.yml: scrape Spring Boot Actuator
scrape_configs:
- job_name: shop
  metrics_path: /actuator/prometheus
  scrape_interval: 15s
  kubernetes_sd_configs:
  - role: pod
    namespaces: { names: [shop] }
  relabel_configs:
  - source_labels: [__meta_kubernetes_pod_label_app]
    regex: shop
    action: keep
  - source_labels: [__meta_kubernetes_pod_ip]
    target_label: __address__
    replacement: ${1}:8081
```

Hardening the execution environment and the management endpoints

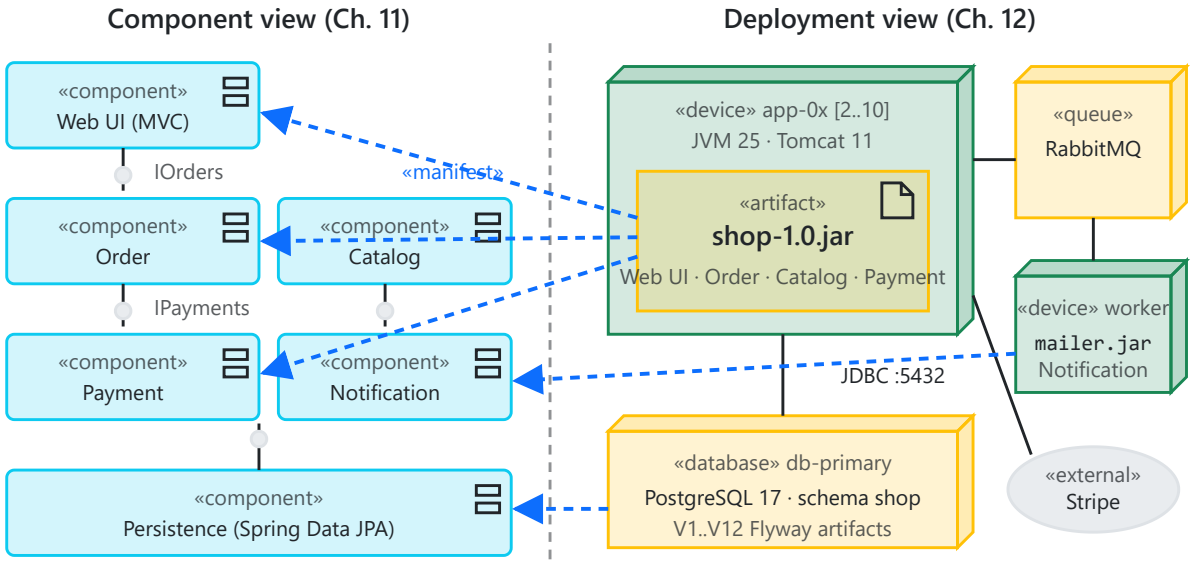
```yaml
# Deployment: securityContext for the shop container
securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities: { drop: [ALL] }
volumeMounts:
- name: tmp
  mountPath: /tmp          # JVM needs a writable /tmp
---
# NetworkPolicy: only ingress-nginx and prometheus may reach shop
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: shop-ingress }
spec:
  podSelector: { matchLabels: { app: shop } }
  ingress:
  - from:
    - namespaceSelector:
        matchLabels: { name: ingress-nginx }
    ports: [{ port: 8080 }]
  - from:
    - podSelector: { matchLabels: { app: prometheus } }
    ports: [{ port: 8081 }]
```

```yaml
# application-prod.yml: Actuator on its own port, minimal exposure
management:
  server:
    port: 8081
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
  endpoint:
    health:
      show-details: never
      probes:
        enabled: true
  metrics:
    tags:
      application: ${spring.application.name}
      instance: ${HOSTNAME:local}
logging:
  structured:
    format:
      console: ecs        # JSON lines for Loki / Elastic
```

```java
// Spring Security 7: management port is internal, but still
// require a role for anything beyond health and prometheus.
@Bean
@Order(1)
SecurityFilterChain actuator(HttpSecurity http) throws Exception {
    http.securityMatcher(EndpointRequest.toAnyEndpoint())
        .authorizeHttpRequests(a -> a
            .requestMatchers(EndpointRequest.to("health", "prometheus"))
                .permitAll()
            .anyRequest().hasRole("OPS"))
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

Signal	Source	Alert when
`http_server_requests_seconds`	Actuator/Micrometer	p95 > 500 ms for 5 min
`hikaricp_connections_pending`	Actuator	> 0 for 1 min (pool exhausted)
`kube_deployment_status_replicas_available`	kube-state-metrics	< 2
`pg_replication_lag`	postgres-exporter	> 30 s

Same system, two views: components (logical) and deployment (physical)



- **Component view**: units of software and their interfaces. No hardware, no ports, no counts.
- **Deployment view**: nodes, artifacts, paths. No interfaces, no classes.
- The **«manifest»** relationship is the bridge: every component must be manifested by at least one artifact, and every artifact must sit on a node. Check both directions.
- Splitting a component onto its own node (Notification → worker) is an *architectural* decision: it adds a communication path (AMQP), a failure mode and a deployment spec.

Component	Artifact	Node
Web UI, Order, Catalog, Payment	`shop-1.0.jar`	app-0x
Notification	`mailer-1.0.jar`	worker
Persistence	jar + `V*.sql`	app-0x + db-primary

Checklist: when is a deployment diagram complete?

#	Check	Question to ask
1	Nodes typed	Every 3D box has «device» or «executionEnvironment» and a real name/size
2	Nesting correct	Artifacts sit inside the environment that executes them
3	Artifacts named with version	`shop-1.0.jar`, `shop:1.0@sha256…`, not "the app"
4	Manifestation traceable	Every Ch. 11 component appears in exactly one or more artifacts
5	Paths labelled	Protocol + port on every solid line; direction of connection
6	Multiplicity	`[2..10]` on scaled nodes; `[*]` on clients
7	Zones and boundaries	DMZ / app / data and where TLS terminates
8	Externals	Third-party systems drawn as «cloud»/«external» with egress rule
9	Deployment specs	Profile, env vars, JVM flags per environment
10	Operations	Monitoring, logs, backups and the failover path are visible

☑ Test: hand the diagram to someone who has never seen the system and ask them to write the firewall rules and the on-call runbook. If they cannot, something is missing.

```
@startuml
skinparam nodesep 30
node "Browser [*]" <<device>> as browser
cloud "CDN + WAF" as cdn
rectangle "DMZ" {
  node "lb-01" <<device>> as lb { artifact "nginx.conf" }
}
rectangle "App zone" {
  node "app-0x [2..10]" <<device>> as app {
    node "JVM 25 / Tomcat 11" <<executionEnvironment>> {
      artifact "shop-1.0.jar" as jar
      artifact "application-prod.yml" <<deployment spec>>
    }
  }
  node "worker-01" <<device>> as worker { artifact "mailer-1.0.jar" }
  node "mon-01" <<device>> as mon { artifact "prometheus.yml" }
}
rectangle "Data zone" {
  database "db-primary\nPostgreSQL 17" as db1
  database "db-standby" as db2
  node "Redis 7" as redis
  queue "RabbitMQ 4" as mq
}
cloud "Stripe" as stripe

browser --> cdn  : HTTPS :443
cdn --> lb       : HTTPS :443
lb --> app       : HTTP :8080
app --> db1      : JDBC :5432 (ssl)
db1 --> db2      : streaming replication
app --> redis    : RESP :6379
app --> mq       : AMQP :5672
mq --> worker    : AMQP :5672
app --> stripe   : HTTPS :443 (egress)
mon ..> app      : scrape :8081
@enduml
```

Best practices and common mistakes

1 Draw for a reader with a task

Operations wants ports and hosts, security wants boundaries, finance wants instance counts. Make one diagram per audience rather than one overloaded poster.

2 Build once, deploy many

One artifact digest travels dev → test → prod. Environments differ only in the deployment specification (profile, env vars, replicas).

3 Keep it in sync with the manifests

Generate or review the diagram from `docker-compose.yml` and the k8s manifests at every release; a stale deployment diagram is worse than none.

✕ Mistake: classes on a deployment diagram

`OrderService` is a component or class, not an artifact. Put the jar it lives in on the node and reference the component through «manifest».

✕ Mistake: unlabelled lines

A line without protocol and port cannot become a firewall rule or a NetworkPolicy. "TCP" alone is not a protocol label.

✕ Mistake: scaled but stateful

Two app nodes with in-memory sessions or local file uploads. If you write `[2..*]`, also show Redis/object storage, or the diagram promises something the code cannot deliver.

✕ Mistake: database as a device

PostgreSQL is software that runs on a node. Draw the node, then the database execution environment (or PlantUML `database`) inside or on it.

✕ Mistake: liveness = readiness

Pointing both probes at a check that includes the database restarts every pod during a DB failover. Liveness: process alive. Readiness: dependencies reachable.

✕ Mistake: secrets in the picture

Never write passwords, keys or connection strings with credentials on a diagram or in git. Show the secret store and the injection path instead.

Chapter quiz 10 questions

1. Which stereotype fits the JVM that runs shop-1.0.jar?

- «device»
- «executionEnvironment»
- «artifact»
- «component»

2. What does a dashed «manifest» arrow connect?

- A node to another node
- An artifact to the component it embodies
- A device to its firewall
- A pod to its Service

3. A solid line between the app server and PostgreSQL is a communication path. What should its label contain?

- The class names involved
- Only the word TCP
- The protocol and the port, e.g. JDBC :5432
- The SQL statements exchanged

4. In UML terms, what is application-prod.yml together with the environment variables?

- A deployment specification
- A communication path
- A device
- A manifestation

5. Which rule protects the data zone in the three-zone design?

- The DMZ may open JDBC connections to the database
- Only app-zone nodes may reach the database, never the DMZ or the Internet
- The database must be placed in the DMZ for latency
- The load balancer terminates JDBC

6. In a containerized deployment, which element is the UML artifact?

- The running container
- The worker node
- The OCI image ghcr.io/itc/shop:1.0
- The containerd daemon

7. What should the readiness probe of the shop pod include that the liveness probe must not?

- The JVM heap size
- Reachability of the database and broker
- The Tomcat thread count
- The image digest

8. Why does the chapter recommend -XX:MaxRAMPercentage=75 instead of a fixed -Xmx?

- It makes the JVM start faster
- It follows the container memory limit automatically
- It disables garbage collection
- It is required by Spring Boot 4

9. What is the correct practice for dev, test and prod artifacts?

- Build a separate jar per environment with baked-in settings
- Build once and promote the same digest, varying only the deployment specification
- Use H2 in production for consistency
- Commit prod secrets into application-prod.yml

10. What does multiplicity [2..10] on the App server node express?

- Ten CPUs per server
- The autoscaling range: at least two, at most ten instances
- Two ports and ten threads
- Ten years of support

WRAP-UP

Summary

- Nodes are 3D boxes: «device» for hardware or VMs, «executionEnvironment» for the JVM, Tomcat, Docker or PostgreSQL that hosts artifacts; nest them to show containment.
- Artifacts are files (jar, image, SQL, yml). «manifest» links them back to components, «deploy» puts them on nodes.
- A deployment specification (profile, env vars, JVM flags, replicas) is what differs between dev, test and prod; the artifact digest does not.
- Communication paths are solid lines labelled with protocol and port (HTTPS 443, JDBC 5432, AMQP 5672); each becomes a firewall rule.
- Zones (DMZ, app, data) and firewalls show trust boundaries; the data zone never accepts traffic from the DMZ.
- Containers: the image is the artifact, the container the execution environment; Kubernetes Deployment, Service and Ingress map to spec, path and entry point.
- Scalability needs stateless nodes plus Redis/broker; availability needs health checks, a standby and a stable address; write the range as multiplicity.
- Monitoring and secrets belong on the diagram as dashed telemetry flows and an injection path from the secret store.
- Every component of Chapter 11 must land in an artifact on a node; use the ten-point checklist before you call the diagram done.

Open Lab-12: 5 tasks + 1 challenge

Course wrap-up: from threads (Ch. 01) through Spring, JPA, REST, security and tests to UML views (Ch. 07-12), you can now build, model and deploy a Java web system end to end.

Back to Chapter 01